2D PATHFINDING - ENEMY AI FOR UNITY

by Indie Devs

For the most up-to-date docs, follow this link.

Video Tutorials 1
Contact 1
Setup **2**
Custom Enemy Movement Script **6**
Scripts **9**
AStarGrid 9
PathRequestManager 11
PathRequest 11
PathResult 11
AStarPathfinding 12
AIMovement 12
SteeringBehaviourManager 13
ObstacleAvoidanceBehaviour 14
EnemyAvoidanceBehaviour 16
SeekBehaviour 17

Video Tutorials

2D PATHFINDING - ENEMY AI for Unity
The Problem with A* Pathfinding in 2D games
2D PATHFINDING - ENEMY AI for Unity: Quick Start

Contact

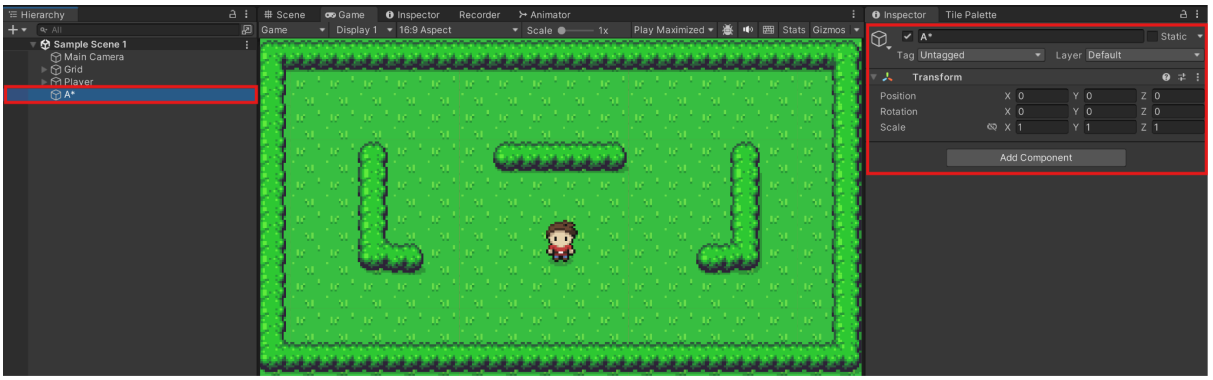
In case you have some questions, suggestions, or need help getting the asset set up, feel free to contact us at:

Email: jozefgocik03@gmail.com
Discord: https://discord.gg/swadkVj2sY

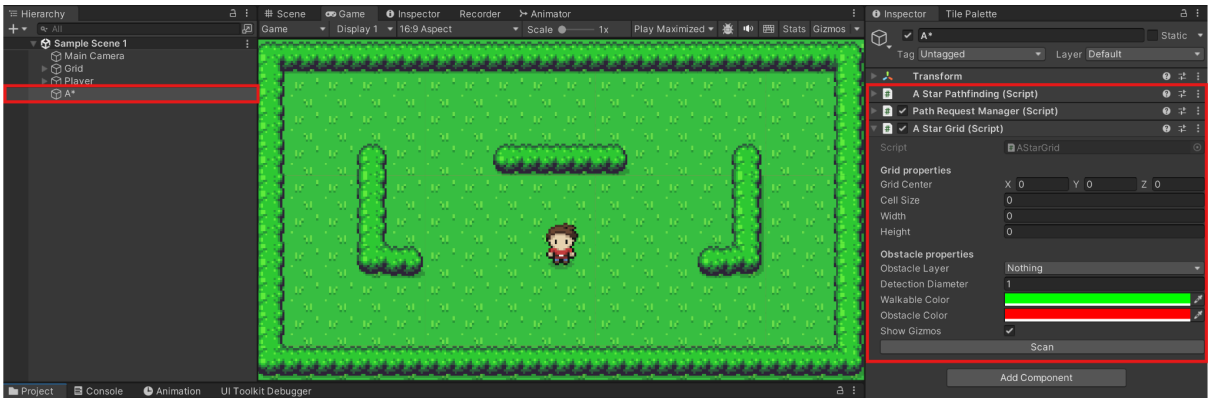
Please rate the asset so Unity recommends it.

Setup

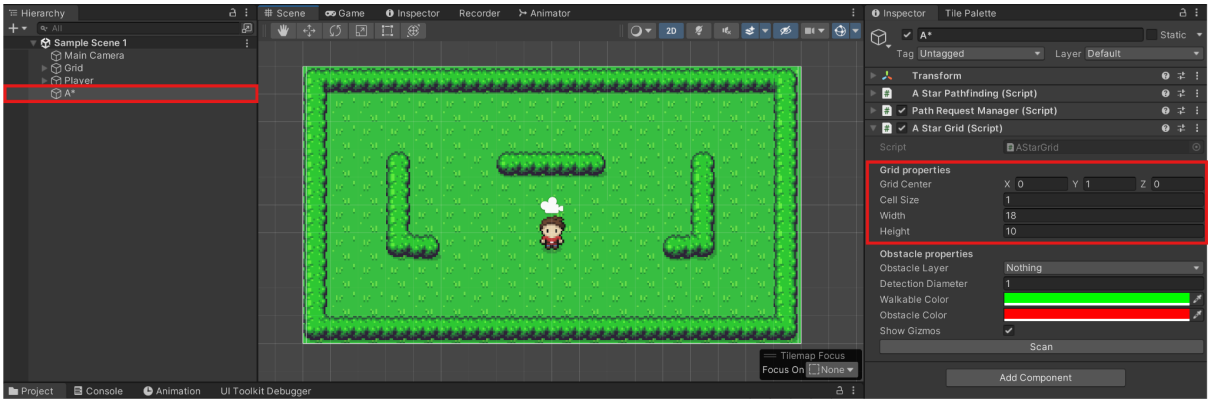
1. Create an empty game object in your scene and give it a name, e.g., A*.



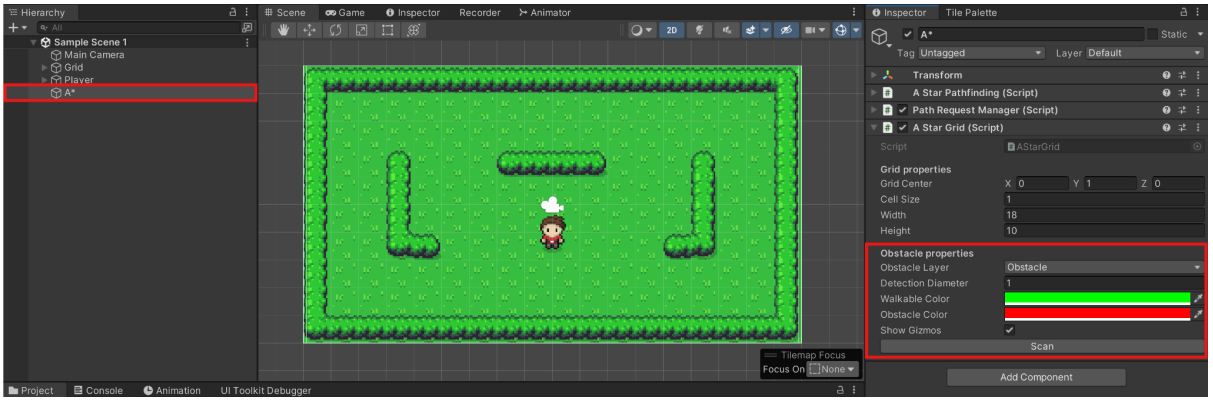
2. Add the AStarGrid and PathRequestManager components to your newly created game object (AStarPathfinding is added automatically).



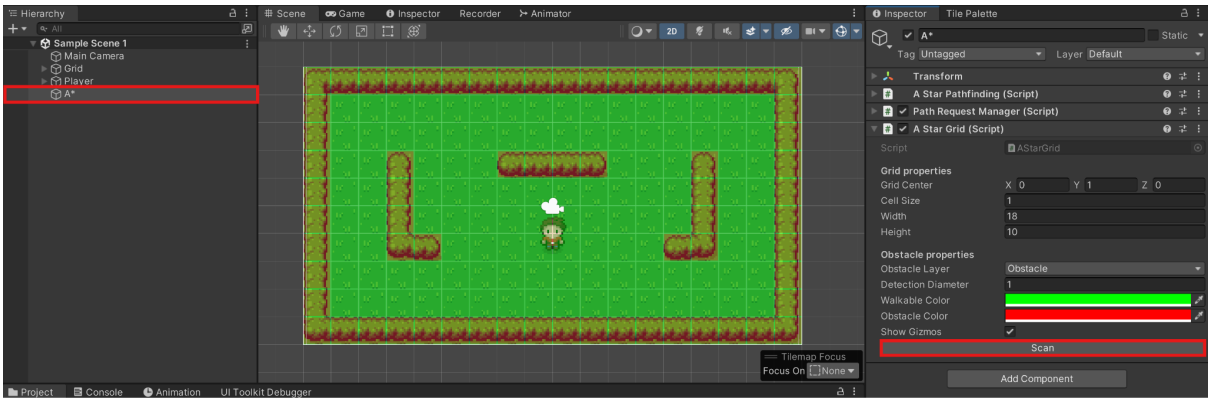
3. Adjust the Grid properties to match the dimensions of your 2D environment.



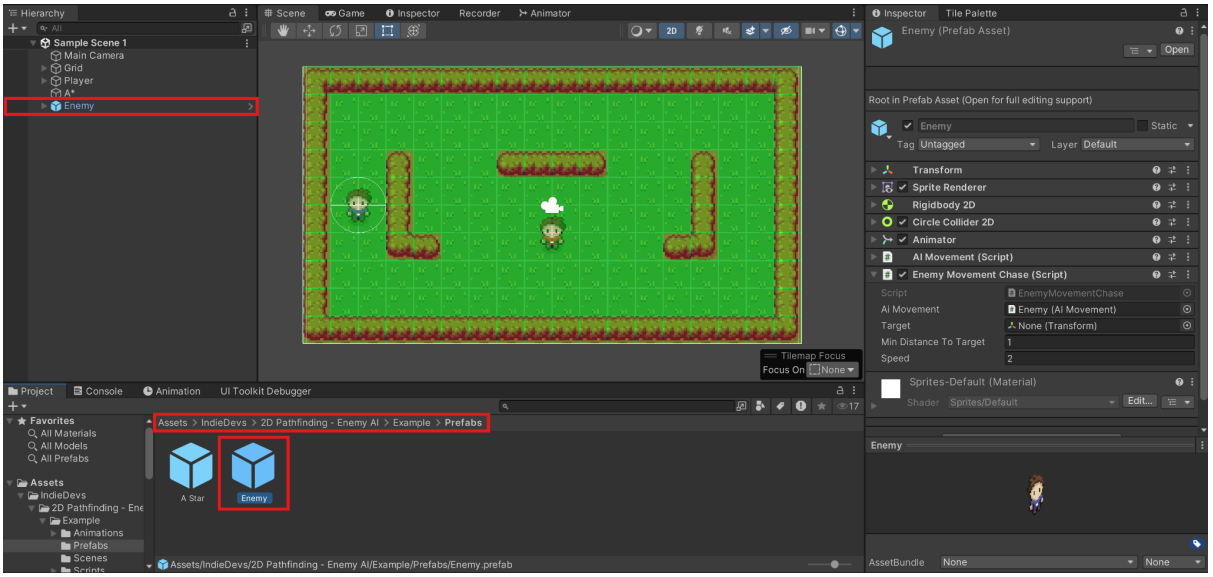
4. Adjust Obstacle properties. Ensure all of your obstacles are on the corresponding Obstacle layer.



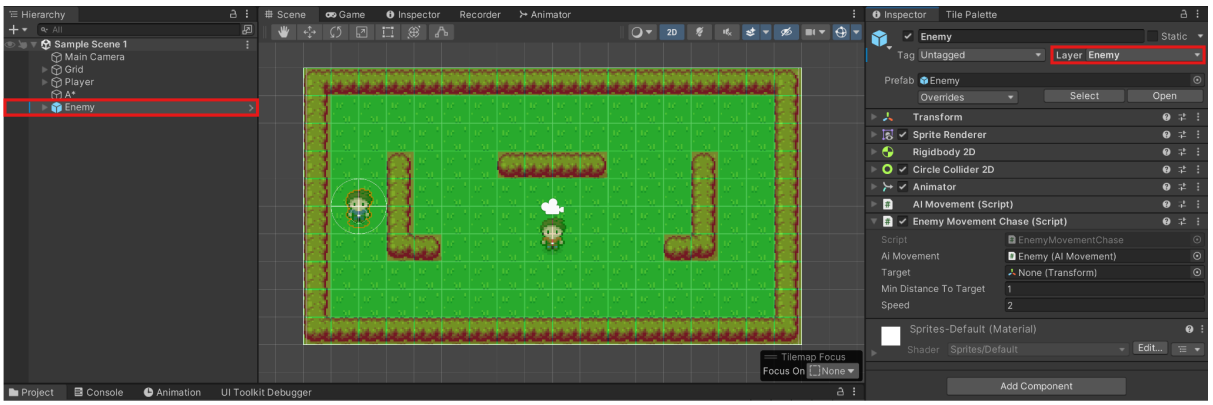
5. Click on the “Scan” button and check that all of the obstacles were properly found.



6. In the Project tab, navigate to Assets/IndieDevs/2D Pathfinding - Enemy AI/Example/Prefabs. Drag and drop the Enemy prefab into your scene.



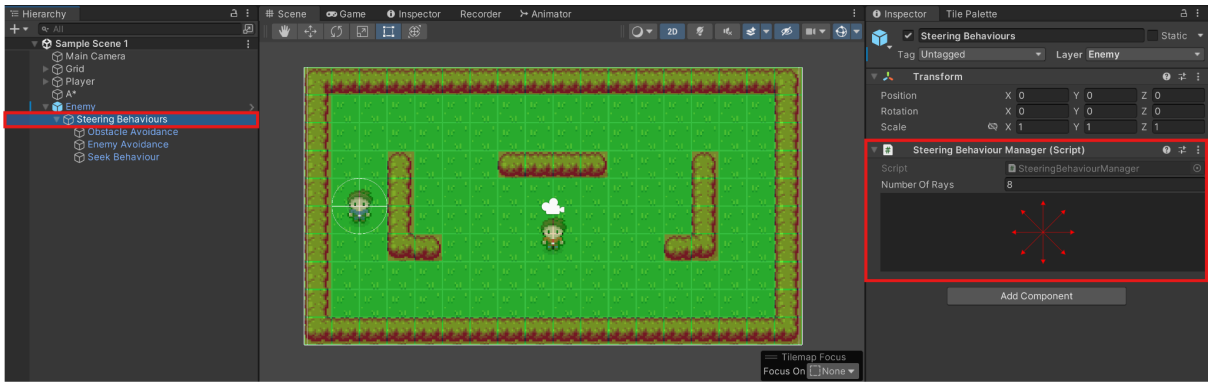
7. Set the layer of the Enemy object (and all of its children) to the corresponding Enemy layer.



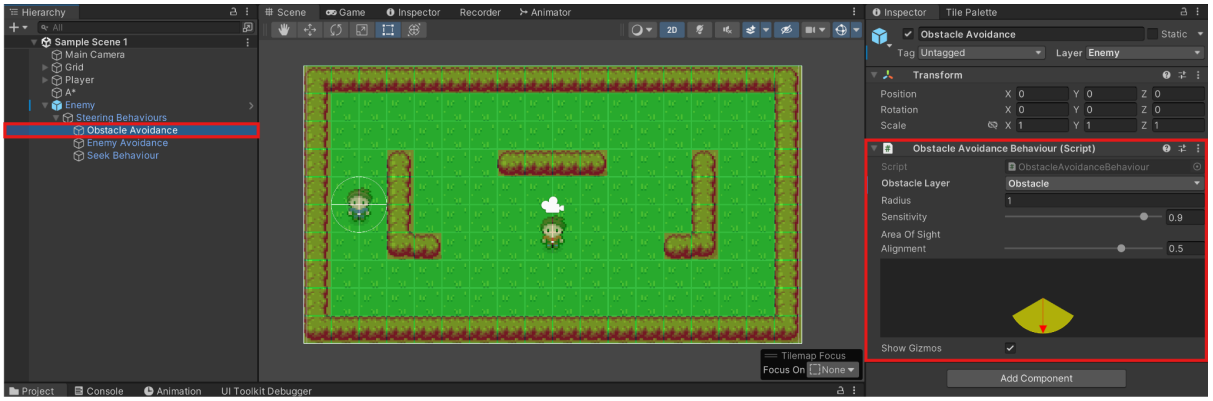
8. Set the Target to be the game object that the Enemy should chase.



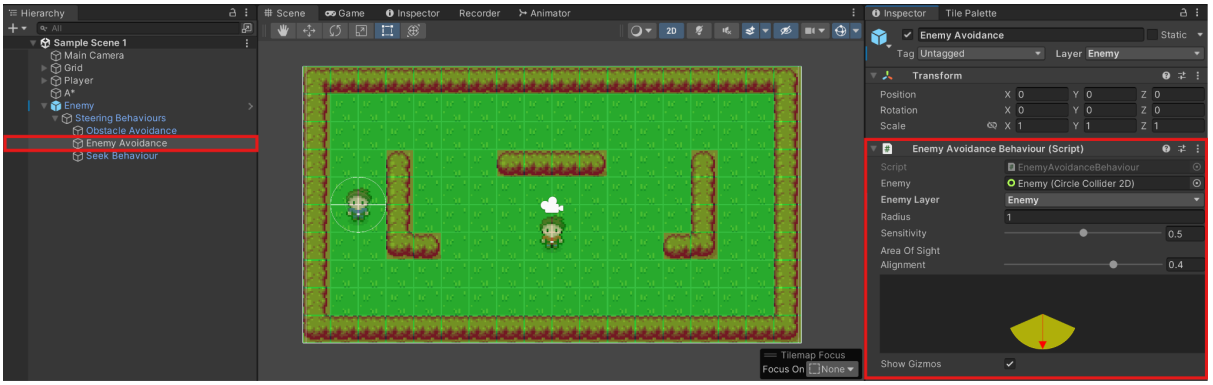
9. Set the number of rays for the Context-based steering behaviour.



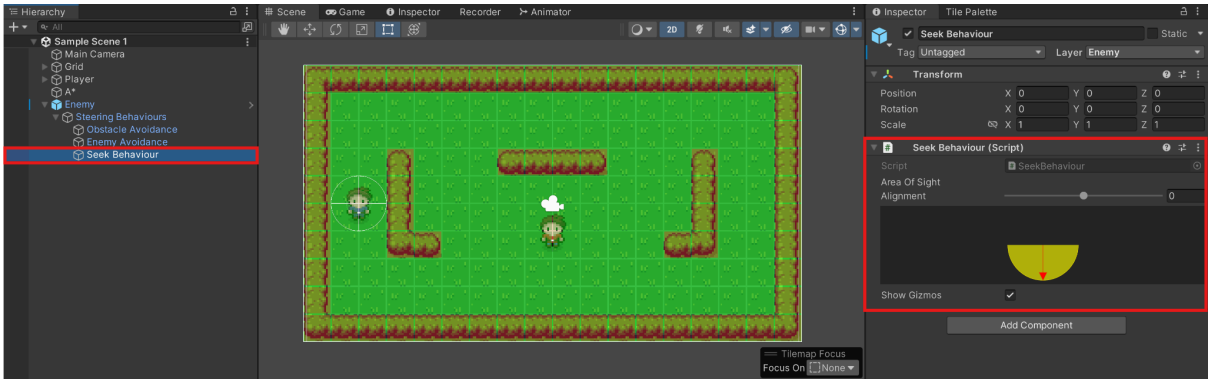
10. Set the Obstacle layer and other properties for the ObstacleAvoidanceBehaviour.



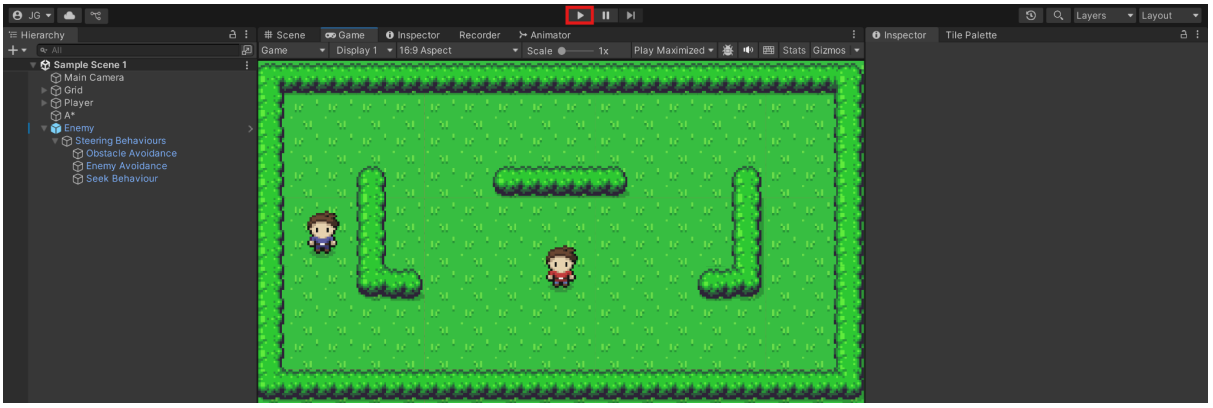
11. Set the Enemy layer and other properties for the EnemyAvoidanceBehaviour.



12. Set up the properties for the SeekBehaviour.



13. Hit “Play” and test the scene.



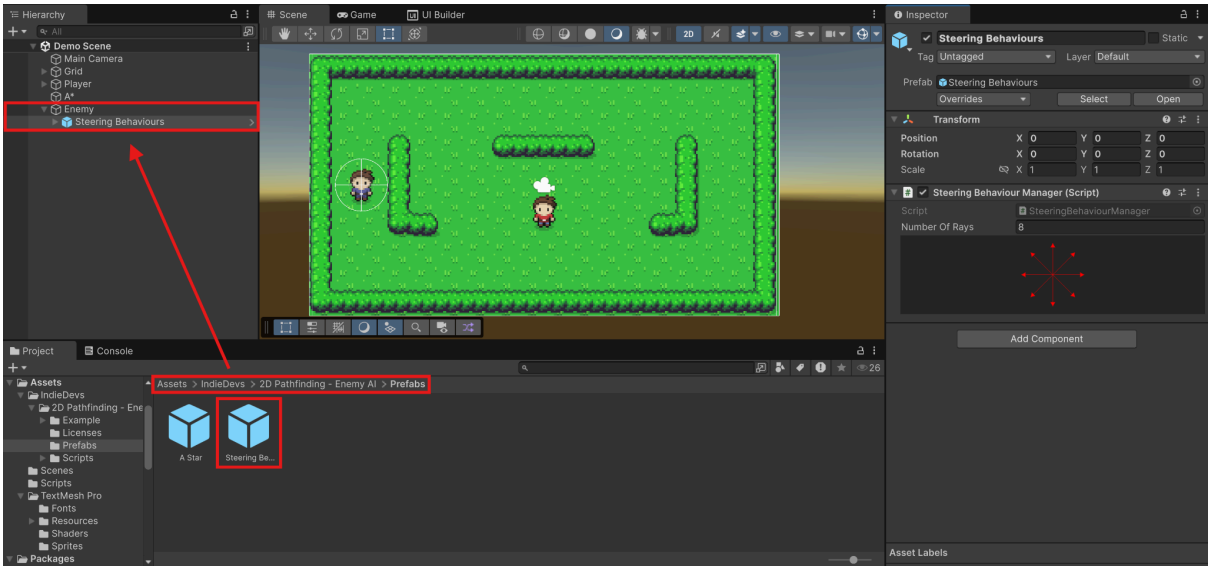
Custom Enemy Movement Script

If you want to use your own script for enemy movement, follow these steps:

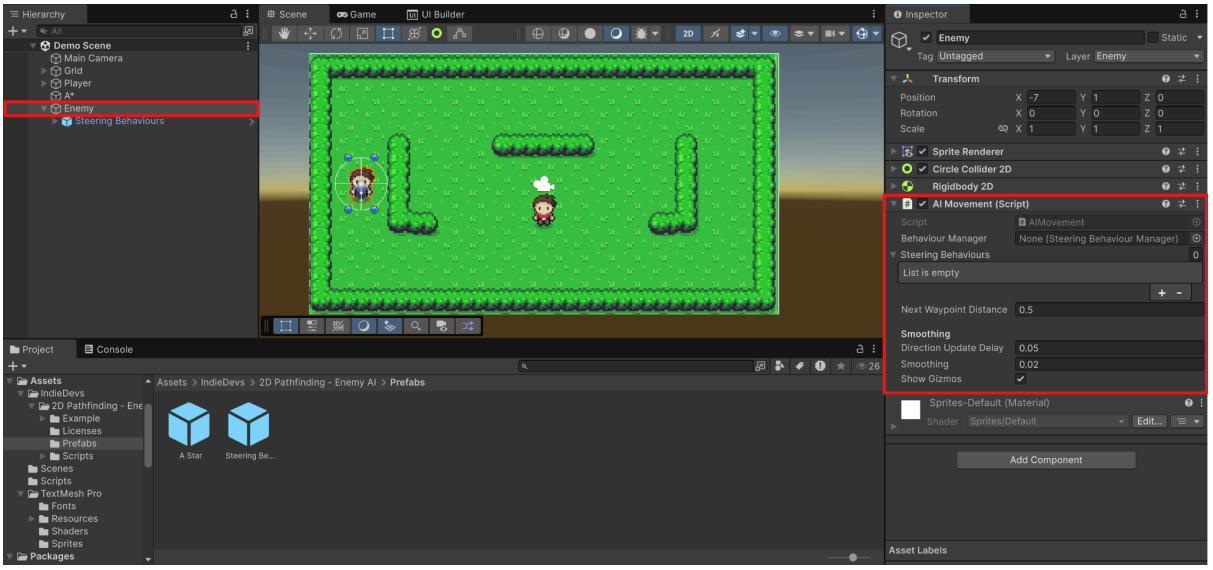
1. Create a game object that represents your enemy. Add SpriteRenderer, Collider2D, and Rigidbody2D components to that game object.



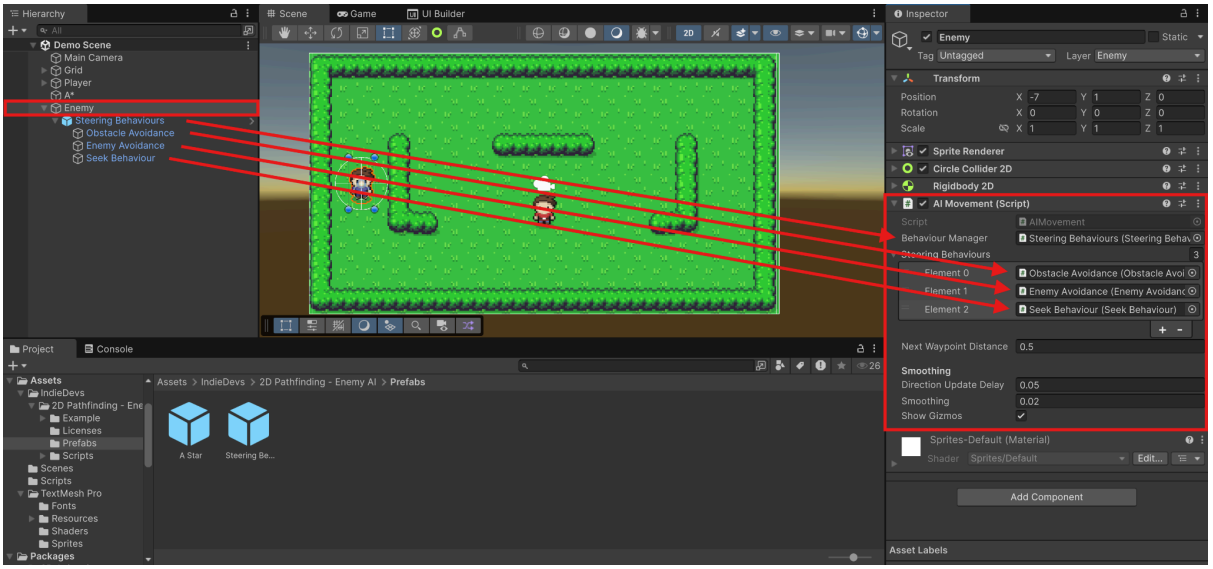
2. In the Project tab, navigate to Assets/IndieDevs/2D Pathfinding - Enemy AI/Prefabs and add the “Steering Behaviours” prefab as a child to the enemy game object.



3. Add the AIMovement script as a component to the enemy game object.



4. Fill in the fields in the AIMovement script with the corresponding references.



5. Create a new C# script for enemy movement. You can use the script below as a starting point.

```
using Pathfinding2D;
using UnityEngine;

public class EnemyMovementExample : MonoBehaviour
{
    public AIMovement aiMovement;
    public Transform target;

    Rigidbody2D rb;

    void Start()
    {
        rb = GetComponent<Rigidbody2D>();

        InvokeRepeating(nameof(RecalculatePath), 0f, 0.1f);
    }

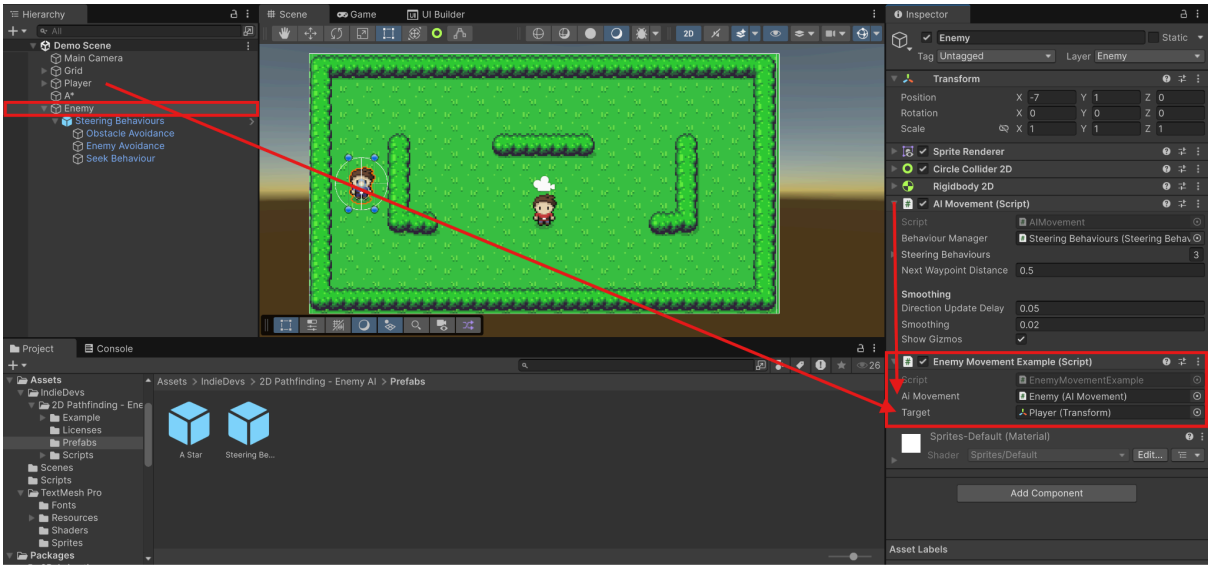
    void FixedUpdate()
    {
        Vector2 finalDirectionToTarget =
aiMovement.GetDirectionToMove(rb.position);

        // INSERT YOUR CODE that moves the ENEMY towards the
finalDirectionToTarget
    }

    void RecalculatePath()
    {
        aiMovement.CalculateShortestPath(rb.position, target.position);
```

```
    }
}
```

6. Add your newly created script (the one for enemy movement) as a component to the enemy game object, and fill in the fields with the corresponding references.



7. Set the layer of the enemy game object and all of its children to the “Enemy” layer.

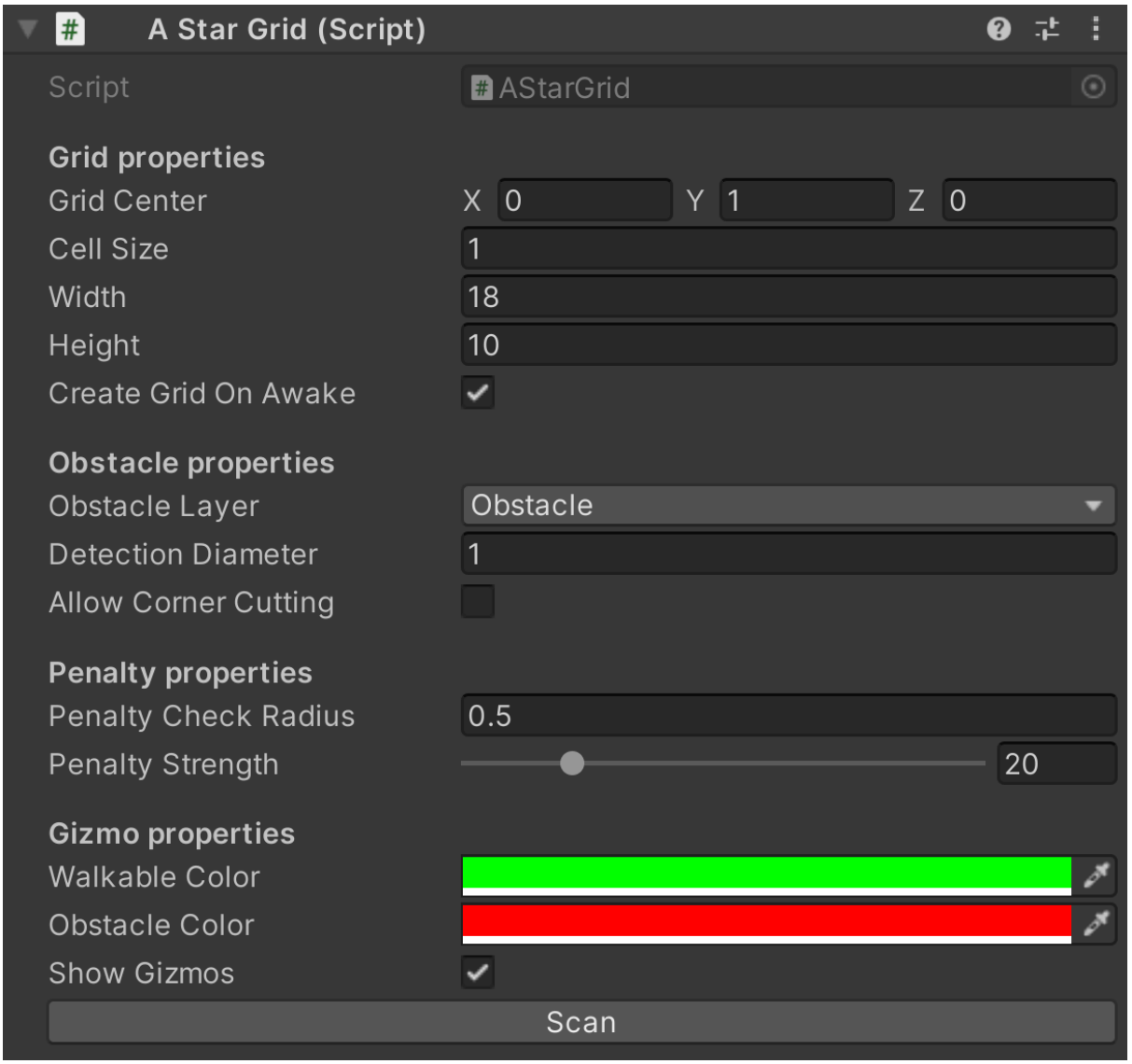


8. Set up the properties of context-based steering by following the steps 9. - 13. from the [Setup].

Scripts

AStarGrid

AStarGrid is responsible for managing properties of the Grid used for calculations in the A* algorithm. It scans the selected area, searches for colliders, and automatically marks the Grid nodes as Walkable or Obstacle.



Fields		
Field Name	**Field Type**	**Field Description**
Grid Center	Vector3	The world-space center position of the grid.
Cell Size	float	The size of each cell in world units.

Width	int	The total number of cells in the horizontal direction.
Height	int	The total number of cells in the vertical direction.
Create Grid On Awake	bool	If true, the grid will be created in the Awake function.
Obstacle Layer	LayerMask	The layer mask used to detect obstacles.
Detection Diameter	float	The diameter (in world units) used for obstacle detection within each cell. Larger values increase sensitivity to obstacles.
Allow Corner Cutting	bool	Determines whether diagonal movement is allowed through tight gaps between two adjacent obstacles.
Penalty Check Radius	float	The radius around each grid cell that is checked for nearby obstacles. Larger values mean the algorithm will “sense” obstacles from farther away and apply penalties to more cells.
Penalty Strength	int	Controls how strongly the A* algorithm avoids cells near obstacles.
Walkable Color	Color	The gizmo color used to display walkable cells.
Obstacle Color	Color	The gizmo color used to display obstacle cells.
Show Gizmos	bool	If true, the grid and its nodes are visualized in the Unity Editor using gizmos.

Public Properties		
Property Name	**Property Type**	**Property Description**
Area	int	The total number of cells in the grid (width * height).

Public Methods		
Method Name	**Return Type**	**Method Description**
CreateGrid()	void	Creates the grid by instantiating nodes and detecting obstacles.
NodeFromWorldPoint(Vector2 worldPosition)	Node	Converts a world-space position into its corresponding grid node.
GetNeighbours(Node node)	List<Node>	Retrieves the 8 neighbouring nodes surrounding a given node.

PathRequestManager

PathRequestManager is responsible for managing pathfinding requests and ensuring they are processed asynchronously and returned safely.

Public Methods		
Method Name	**Return Type**	**Method Description**
RequestPath(PathRequest request)	static void	Submits a new pathfinding request using the default pathfinding instance.
RequestPath(PathRequest request, AStarPathfinding pathfinding)	static void	Submits a new pathfinding request using a specified pathfinding instance.

PathRequest

PathRequest is a struct data type that holds information required for the calculation of the shortest path.

Public Methods		
Method Name	**Return Type**	**Method Description**
PathRequest(Vector3 start, Vector3 end, bool reachClosestWalkableNode Action<Vector3[], bool> callback)	PathRequest	Initializes a new PathRequest instance.

PathResult

PathResult is a struct data type that holds information about the calculated shortest path.

Public Methods		
Method Name	**Return Type**	**Method Description**
PathResult(Vector3[] path, bool success, Action<Vector3[], bool> callback)	PathResult	Initializes a new PathResult instance.

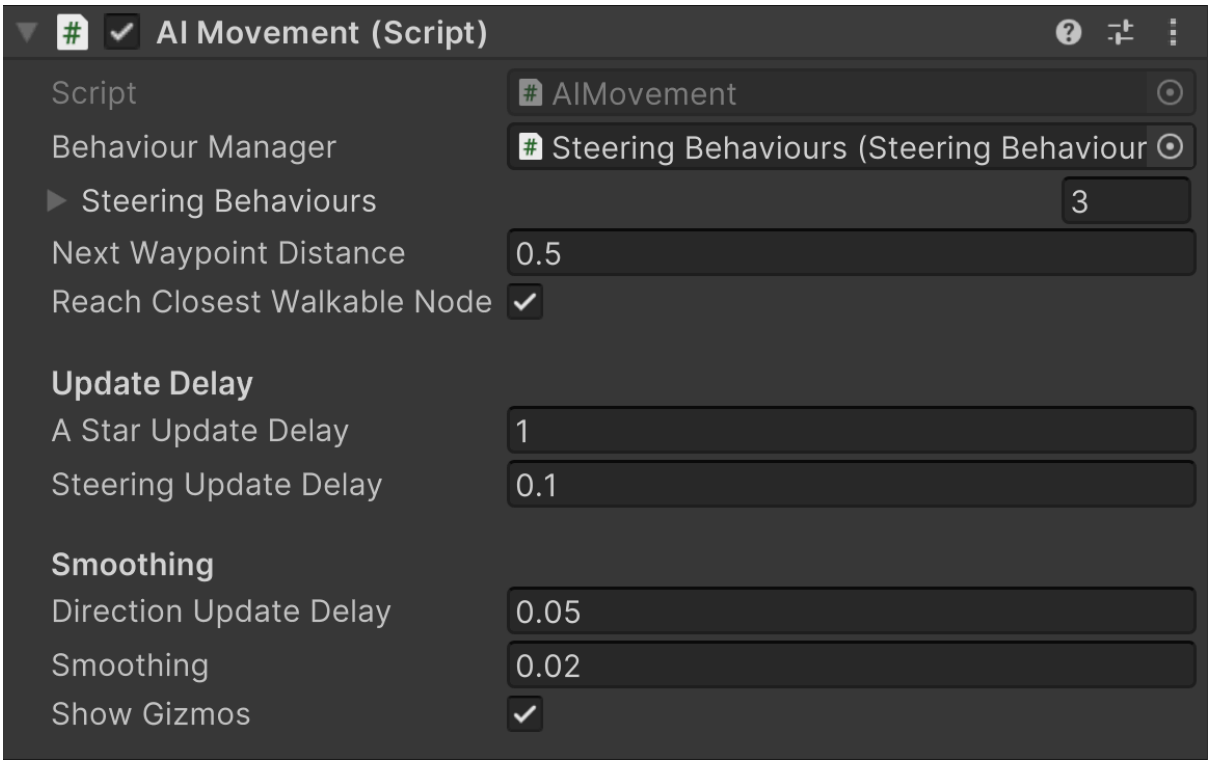
AStarPathfinding

AStarPathfinding implements the A* pathfinding algorithm for navigating a 2D grid. It uses AStarGrid nodes to calculate the shortest path between two world positions.

Public Methods		
Method Name	**Return Type**	**Method Description**
FindPath(PathRequest pathRequest, Action<PathResult> callback)	void	Finds the shortest path between two points using the A* algorithm.

AIMovement

AIMovement handles enemy movement using a combination of A* pathfinding and steering behaviours for more natural navigation.



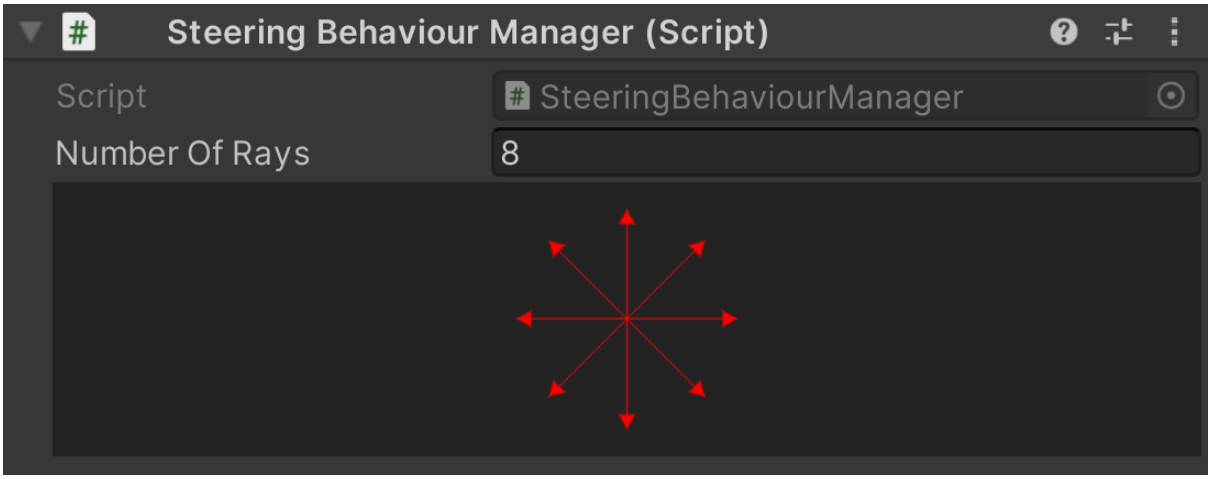
Fields		
Field Name	**Field Type**	**Field Description**
Behaviour Manager	SteeringBehaviourManager	A reference to the Steering Behaviour Manager of the enemy.
SteeringBehaviours	List<SteeringBehaviour>	List of references to all steering behaviors used by the enemy.
Next Waypoint Distance	float	Minimum distance from the current waypoint required to move to the next waypoint.
Reach Closest Walkable Node	bool	When enabled, the pathfinding system will attempt to find the closest reachable walkable node to the requested target position if the target itself is unreachable or lies on an obstacle.
A Star Update Delay	float	Minimum delay between A* path recalculations.
Steering Update Delay	float	Minimum delay between Context-based steering recalculations.
Direction Update Delay	float	Minimum delay (in seconds) between direction recalculations. Helps prevent jittery movement.
Smoothing	float	Number of seconds used for smooth transitions between directions.
Show Gizmos	bool	If true, the path will be drawn as gizmos in the Unity Editor.

Public Properties		
Property Name	**Property Type**	**Property Description**
IsPathValid	bool	Checks whether the current path is valid.

Public Methods		
Method Name	**Return Type**	**Method Description**
CalculateShortestPath(Vector3 from, Vector3 to)	void	Calculates a new shortest path between two points using the default pathfinding instance, and updates steering behaviours.
CalculateShortestPath(Vector3 from, Vector3 to, AStarPathfinding pathfinding)	void	Calculates a new shortest path between two points using the specified pathfinding instance, and updates steering behaviours.
GetDirectionToMove(Vector3 from)	Vector2	Returns the movement direction the enemy should take, based on pathfinding and steering behaviour calculations.

SteeringBehaviourManager

SteeringBehaviourManager combines multiple steering behaviors to determine the final movement direction.



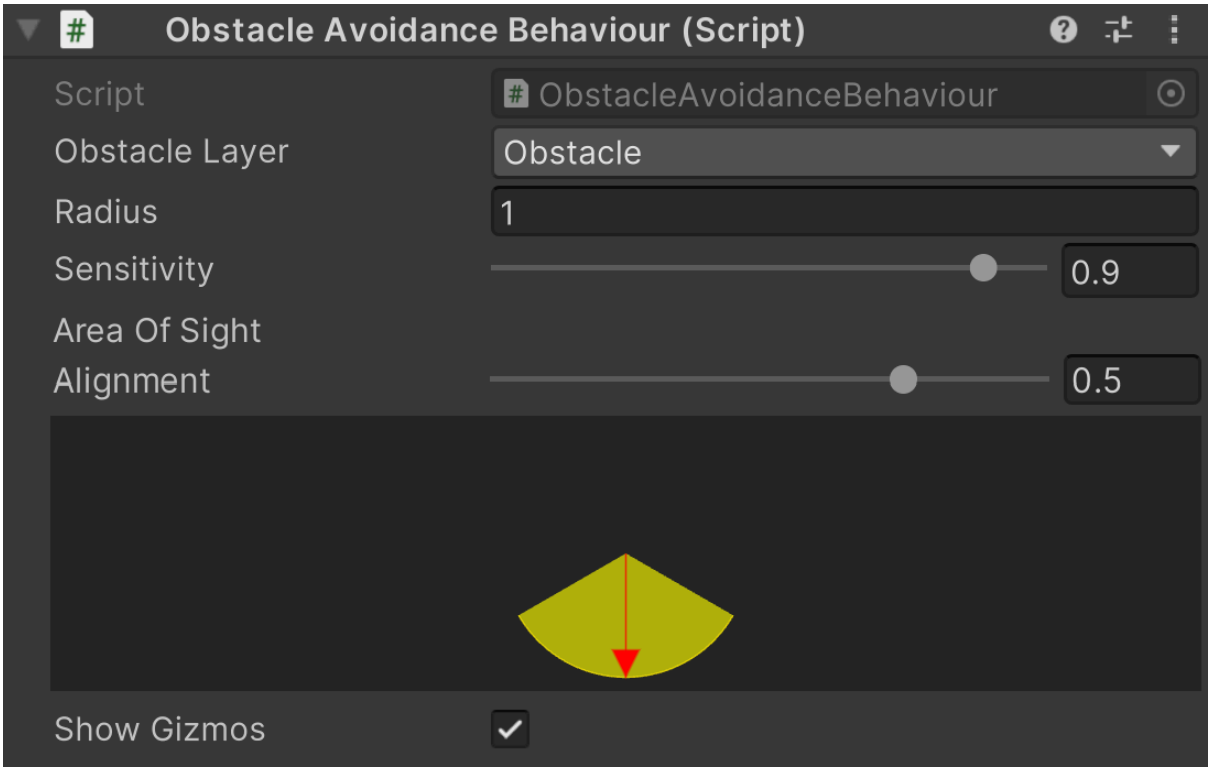
Fields		
Field Name	**Field Type**	**Field Description**
Number Of Rays	int	The number of rays used to sample possible movement directions. Higher values increase accuracy at the cost of performance.

Public Properties		
Property Name	**Property Type**	**Property Description**
Directions	Vector2[]	An array of evenly distributed direction vectors around a circle, based on the Number Of Rays.

Public Methods		
Method Name	**Return Type**	**Method Description**
GetFinalDirection(List<SteeringBehaviour> behaviours, Vector2 desiredDirection)	Vector2	Calculates the final movement direction for an enemy by combining the influence of multiple steering behaviours.

ObstacleAvoidanceBehaviour

ObstacleAvoidanceBehaviour calculates which directions the enemy should eliminate in order to avoid obstacles.



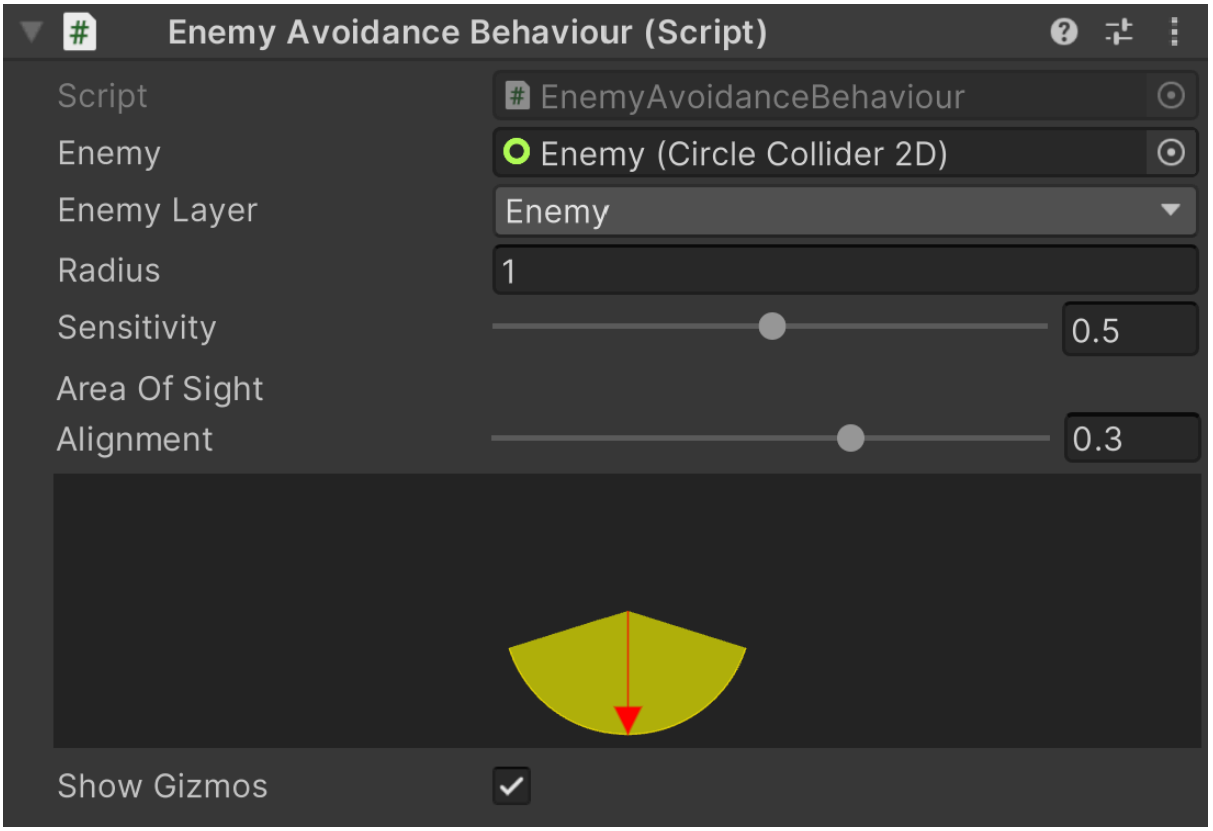
Fields		
Field Name	**Field Type**	**Field Description**

Obstacle Layer	LayerMask	The layer mask used to detect obstacles.
Radius	float	The radius of a circle used for the detection of the surrounding obstacles.
Sensitivity	float	The number representing how sensitive the enemy is to the obstacles. The higher the number, the more strictly the enemy will avoid obstacles.
Area Of Sight	float	Area of the circle used for the detection of obstacles. All the directions that are within this area will be taken into consideration while calculating the final direction.
Show Gizmos	bool	If true, the radius and danger rays will be drawn as gizmos in the Unity Editor.

Public Methods		
Method Name	**Return Type**	**Method Description**
Detect()	void	This method looks for surrounding obstacles within a specified radius.
GetSteering(SteeringBehaviourManager behaviourManager, Vector2 desiredDirection)	(float[] danger, float[] interest)	The method calculates which direction should be eliminated in order to avoid obstacles.

EnemyAvoidanceBehaviour

EnemyAvoidanceBehaviour calculates which directions the enemy should eliminate in order to avoid other enemies.



Fields		
Field Name	**Field Type**	**Field Description**
Enemy	Collider2D	The collider of the enemy that has the EnemyAvoidanceBehaviour script attached. It ensures that the enemy doesn't avoid itself.
Enemy Layer	LayerMask	The layer mask used to mark enemies.
Radius	float	The radius of a circle used for the detection of the surrounding enemies.
Sensitivity	float	The number representing how sensitive the enemy is to other enemies. The higher the number, the more strictly the enemy will avoid other enemies.
Area Of Sight	float	Area of the circle used for the detection of other enemies. All the directions

		that are within this area will be taken into consideration while calculating the final direction.
Show Gizmos	bool	If true, the radius and danger rays will be drawn as gizmos in the Unity Editor.

Public Methods		
Method Name	**Return Type**	**Method Description**
Detect()	void	This method looks for surrounding enemies within a specified radius.
GetSteering(SteeringBehaviourManager behaviourManager, Vector2 desiredDirection)	(float[] danger, float[] interest)	The method calculates which direction should be eliminated in order to avoid enemies.

SeekBehaviour

SeekBehaviour calculates which directions the enemy should take in order to get to the target as soon as possible.



Fields		
Field Name	**Field Type**	**Field Description**
Area Of Sight	float	Area of the circle used for

		searching the best direction. All the directions that are within this area will be taken into consideration while calculating the final direction.
Show Gizmos	bool	If true, the interest rays will be drawn as gizmos in the Unity Editor.

Public Methods		
Method Name	**Return Type**	**Method Description**
Detect()	void	-
GetSteering(SteeringBehaviourManager behaviourManager, Vector2 desiredDirection)	(float[] danger, float[] interest)	The method calculates which direction should be taken in order to get to the target.